오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

JavaScript 기초

DAY 11

연산자와 조건문

담아 둔 값으로 계산하고 비교합니다.
비교한 결과에 따라 다른 코드를 실행하는 법을 배웁니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **산술 · 대입 · 비교 · 논리 연산자로 값을 계산하고 비교할 수 있다**

   % 와 ** , === 와 == 의 차이, && · || 가 돌려주는 값을 읽습니다.

2. **if · else if · else와 switch로 갈 길을 나눌 수 있다**

   truthy · falsy를 알고, 값 하나를 고를 때는 삼항 연산자를 씁니다.

3. **조건식의 실수를 찾아 고칠 수 있다**

   = 와 === , 조건의 순서, || 뒤의 비교식, 빠진 break를 점검합니다.

오늘의 결과물

로그인 상태에 따라 인사가 바뀌는 login.js와 점수를 넣으면 등급이 나오는 grade.js

ROADMAP

오늘의 흐름

01

산술과 대입

+ - * / % **
우선순위와 +=

02

비교와 논리

=== · && · ||
truthy · falsy

03

조건문

if · else if · switch
실습 01

실습 포함

04

삼항과 오류 점검

조건식 실수 찾기
실습 02

실습 포함

앞의 두 섹션은 값을 계산하고 비교하는 연산자, 뒤의 두 섹션은 그 결과로 갈 길을 나누는 조건문입니다.
오늘도 화면은 바꾸지 않고 모든 결과를 Console에서 봅니다.

어제 배운 것 — 값 담기와 확인

```
// DAY 10에서 한 것
const price = 12000;          // const : 다시 담지 않는 값
let count = 1;                // let : 다시 담는 값
count = 2;                    // = 는 '담기'
console.log(typeof price);    // "number"

console.log("12000" + 3000);  // 120003000 ← 문자열이면 이어 붙임
```

오늘은 계산과 비교

담아 둔 값으로 더하고,
나누고, 같은지 묻습니다.

그리고 갈림길

비교한 결과(참·거짓)에
따라 다른 코드를
실행합니다.

되짚기 · 02

조건문의 재료, 참·거짓

```
let isMember = true;
console.log(typeof isMember);   // "boolean"

console.log(10 < 20);           // true  ← 비교한 결과도 boolean
console.log(10 > 20);           // false
```

비교하면 boolean

크다, 작다, 같다를 물으면
true 아니면 false가
돌아옵니다.

오늘 if가 씁니다

if는 이 true · false를 보고
{ } 안을 실행할지
정합니다.

준비

오늘의 셋팅

```
Day11/                    ← 같은 저장소 안에 만듭니다
  login.html              ← login.js 를 연결합니다 (실습 01)
  login.js
  grade.html              ← grade.js 를 연결합니다 (실습 02)
  grade.js
$ git status
/* 오늘도 main에 바로 커밋합니다 */
```

짧은 예제는 Console

슬라이드의 한두 줄 코드는
Console에 바로 쳐서
결과를 확인합니다.

실습은 파일로

로그인 인사와 등급 계산은
.js 파일에 쓰고 값을 바꿔
가며 새로고침합니다.

01

산술과 대입

연산자와 피연산자 · 나머지와 거듭제곱 · 증가와 감소 · 우선순위 · 대입

값을 받아 새 값을 만드는 연산자



피연산자가 둘이면 이항, 하나면 단항 연산자

10 + 20 은 이항 연산자, count++ 나 -num 처럼 피연산자가 하나면 단항 연산자입니다.

이항 산술 연산자 여섯 개

연산자	뜻	예	결과
`+`	더하기	`10 + 20`	`30`
`-`	빼기	`20 - 10`	`10`
`*`	곱하기	`10 * 20`	`200`
`/`	나누기	`10 / 4`	`2.5`
`%`	나머지	`10 % 3`	`1`
`**`	거듭제곱	`2 ** 3`	8 (2를 세 번 곱함)

나누어떨어지지 않으면 소수까지 나옵니다

10 / 3 의 결과는 3.3333333333333335 입니다. 몫과 나머지를 따로 원하면 % 를 함께 씁니다.

01 · 나머지

생각보다 자주 쓰는 나머지 %

```
console.log(7 % 2);      // 1   ← 2로 나눈 나머지가 1이면 홀수
console.log(8 % 2);      // 0   ← 0이면 짝수
console.log(17 % 5);     // 2   ← 17개를 5개씩 묶으면 2개 남음
console.log(125 % 60);   // 5   ← 125분은 2시간 5분

console.log(2 ** 10);    // 1024 ← 2를 열 번 곱함
```

짝수 · 홀수 판별

n % 2 가 0이면 짝수,
1이면 홀수입니다.
조건문에서 다시 씁니다.

묶고 남는 것

몇 개씩 묶고 남는 수,
분을 시간으로 바꾸고
남는 분을 셀 때 씁니다.

01 · 증가와 감소

++ 는 1 더하기, -- 는 1 빼기

```
let count = 10;
count++;          // 11 ← count = count + 1 과 같음
count--;          // 10 ← count = count - 1 과 같음

// 다른 식 안에 넣으면 차이가 생깁니다 (1기 자료의 예)
let num = 10;
let a = num++;    // a는 10, num은 11 ← 담은 뒤 더하기
let num2 = 10;
let b = ++num2;   // b는 11 ← 더한 뒤 담기
```

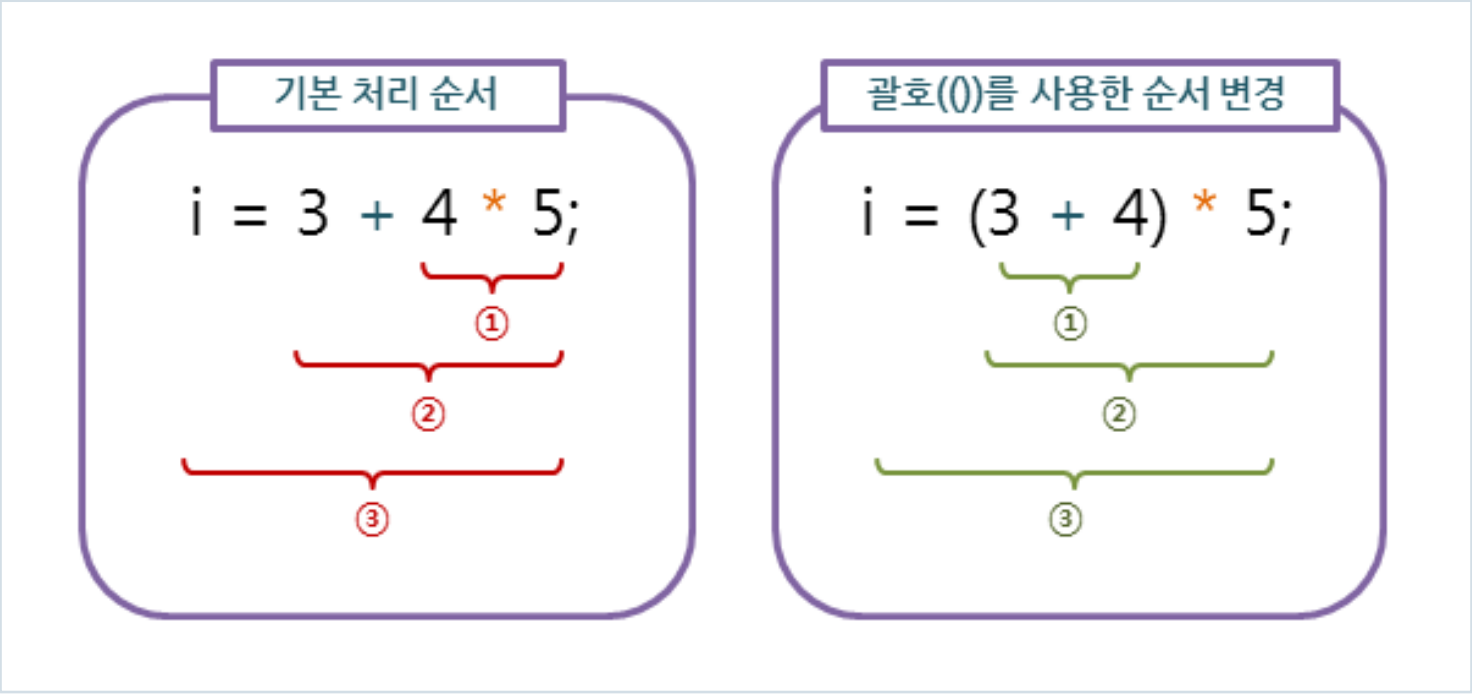
한 줄에 따로 씁니다

count++; 처럼 혼자 쓰면 앞에 붙이든 뒤에 붙이든 결과가 같습니다.

숫자에는 못 씁니다

10++ 처럼 숫자에 붙이면 SyntaxError가 납니다. const 변수에도 못 씁니다.

곱하기가 먼저, 괄호가 가장 먼저



왼쪽은 기본 순서, 오른쪽은 괄호로 순서를 바꾼 것 — 원 숫자가 계산하는 차례

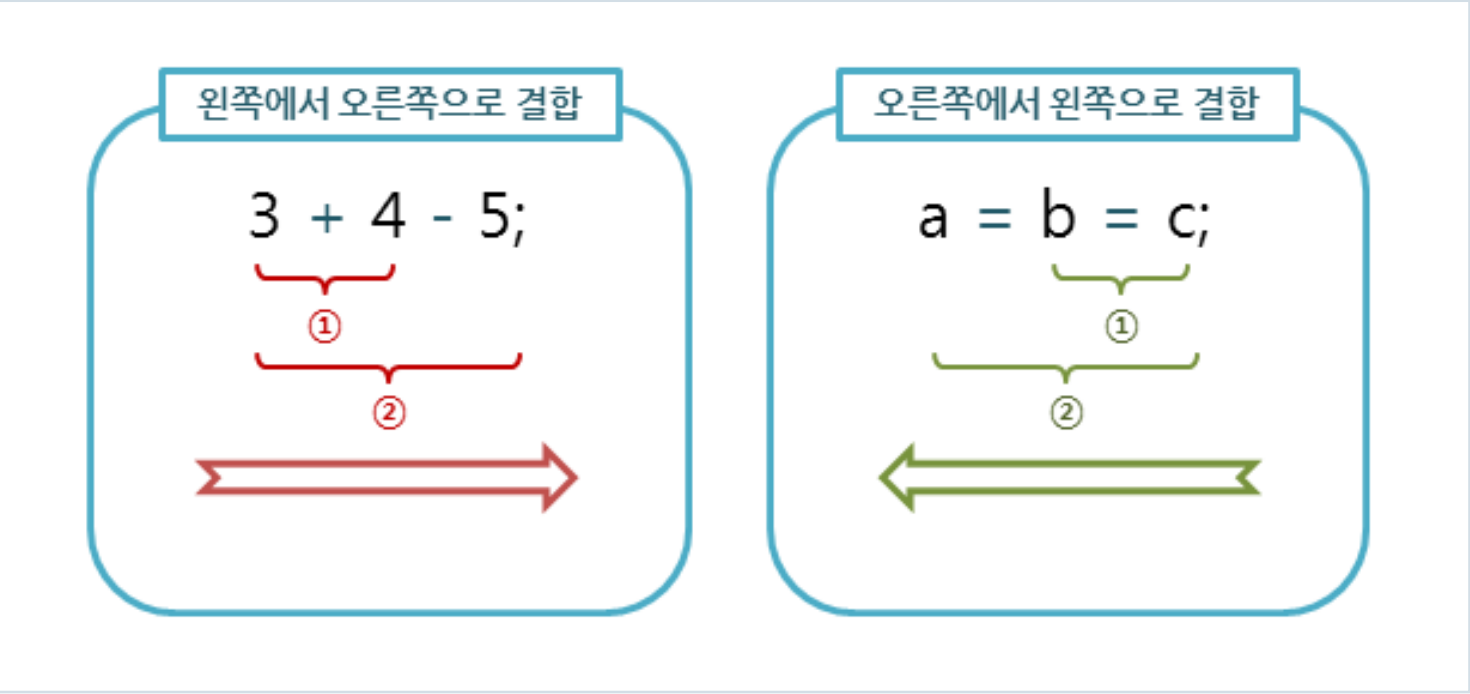
3 + 4 * 5 → 23

* 를 먼저 계산합니다.
4 * 5 = 20, 3 + 20 = 23

(3 + 4) * 5 → 35

괄호 안을 먼저 계산합니다.
3 + 4 = 7, 7 * 5 = 35

순위가 같으면 왼쪽부터 계산



왼쪽은 3 + 4 - 5 를 왼쪽부터, 오른쪽은 a = b = c 를 오른쪽부터 계산

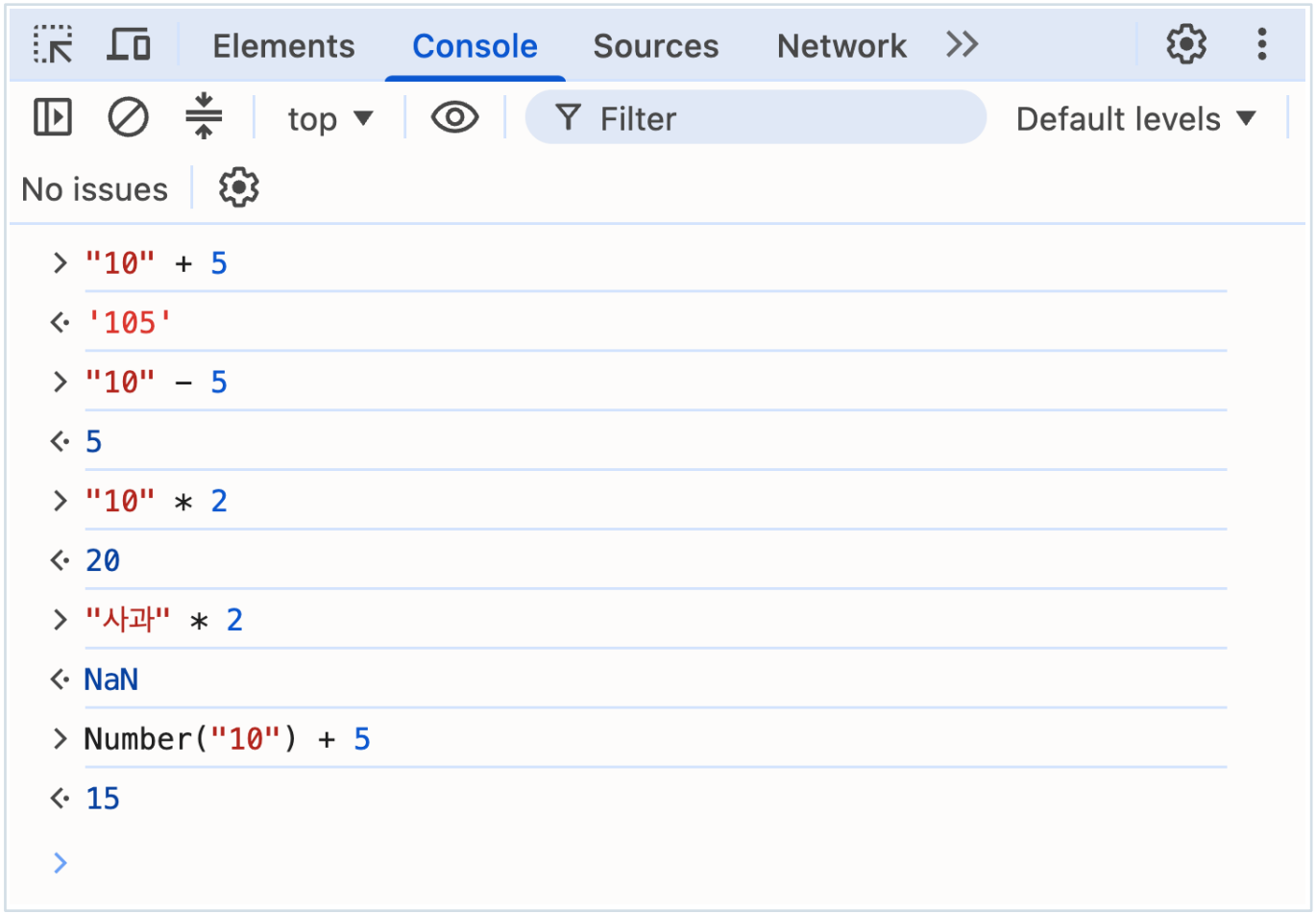
3 + 4 - 5 → 2

+ 와 - 는 순위가 같아
왼쪽부터 계산합니다.
3 + 4 = 7, 7 - 5 = 2

a = b = c

대입은 오른쪽부터.
b에 c를 담고, 그 값을
다시 a에 담습니다.

+ 만 이어 붙이기, 나머지는 숫자로 계산



+ 는 이어 붙이기

"10" + 5 는 "105".
결과에 따옴표가 붙어
있으면 문자열입니다.

- * / 는 숫자로

"10" - 5 는 5입니다.
숫자로 못 바꾸면 NaN,
Number()로 먼저 바꿉니다.

01 · 대입

계산과 담기를 한 번에 하는 연산자

연산자	예 (x에는 10)	같은 뜻	결과
+=	`x += 5`	`x = x + 5`	15
-=	`x -= 5`	`x = x - 5`	5
*=	`x *= 5`	`x = x * 5`	50
/=	`x /= 5`	`x = x / 5`	2
%=	`x %= 3`	`x = x % 3`	1
**=	`x **= 2`	`x = x ** 2`	100

오른쪽을 계산한 뒤 왼쪽 변수에 다시 담습니다

01 · 대입

값을 쌓을 때 쓰는 +=

```
let total = 0;
total += 12000;            // 12000 ← 티셔츠
total += 3000;             // 15000 ← 배송비
console.log(total);       // 15000

let message = "안녕";
message += "하세요";
console.log(message);     // 안녕하세요
```

합계 쌓기

0에서 시작해 값을
하나씩 더해 갑니다.
DAY 12 반복문에서 다시.

let으로 만듭니다

계속 다시 담으므로
const로 만들면
TypeError가 납니다.

여기까지 정리

1	**산술**	+ - * / % ** — 나누기는 소수까지, % 는 나머지
2	**문자열**	+ 는 문자열이면 이어 붙이고, - * / 는 숫자로 바꿔 계산
3	**우선순위**	곱하기·나누기가 먼저, 괄호가 가장 먼저
4	**대입**	x += 5 는 x = x + 5 — 값을 쌓을 때

다음은 값을 비교하는 연산자입니다

02

비교와 논리

=== 와 == · && · || · ! · 값을 돌려주는 && · || · truthy · falsy

비교 연산자 여덟 개

연산자	뜻	예	결과
`===`	값과 자료형이 모두 같다	`10 === "10"`	`false`
`!==`	값이나 자료형이 다르다	`10 !== "10"`	`true`
`==`	자료형을 바꿔서 같다	`10 == "10"`	`true`
`!=`	자료형을 바꿔서 다르다	`10 != "10"`	`false`
`<` `<=`	작다 · 작거나 같다	`10 < 10` `10 <= 10`	`false` `true`
`>` `>=`	크다 · 크거나 같다	`10 > 10` `10 >= 10`	`false` `true`

비교한 결과는 언제나 true 아니면 false입니다

같은지 묻는 ===

```
console.log(10 == "10");          // true  ← 자료형을 맞춰서 비교
console.log(10 === "10");         // false ← 숫자와 문자열은 다름
console.log(0 == "");             // true
console.log(0 === "");            // false
console.log(null == undefined);   // true
console.log(null === undefined);  // false
```

== 는 바꿔서 비교

자료형이 달라도 같은 값으로 바꿀 수 있으면 true가 됩니다.

=== 를 씁니다

자료형까지 같아야 true. 이 과정은 === 와 !== 만 씁니다.

02 · 논리

조건 여러 개를 묶는 && · || · !

AND · 그리고

&&

둘 다 참이어야 참.
age >= 20 && hasTicket
나이도 되고 표도 있어야

OR · 또는

||

하나만 참이어도 참.
isAdmin || isOwner
관리자거나 주인이거나

NOT · 아니다

!

참과 거짓을 뒤집습니다.
!isLoggedIn
로그인하지 않았으면

! 는 바로 뒤의 값 하나에 붙습니다

식 전체를 뒤집을 때는 !(...) 처럼 괄호로 감쌉니다.

비교 먼저, 묶기는 그다음

```
const age = 25;
const hasTicket = true;

console.log(age >= 20 && hasTicket);   // true  ← 둘 다 참
console.log(age < 20 || hasTicket);    // true  ← 하나가 참
console.log(!hasTicket);               // false ← 뒤집기

// 괄호로 묶는 범위에 따라 결과가 다릅니다
console.log(!true && false);           // false
console.log(!(true && false));         // true
```

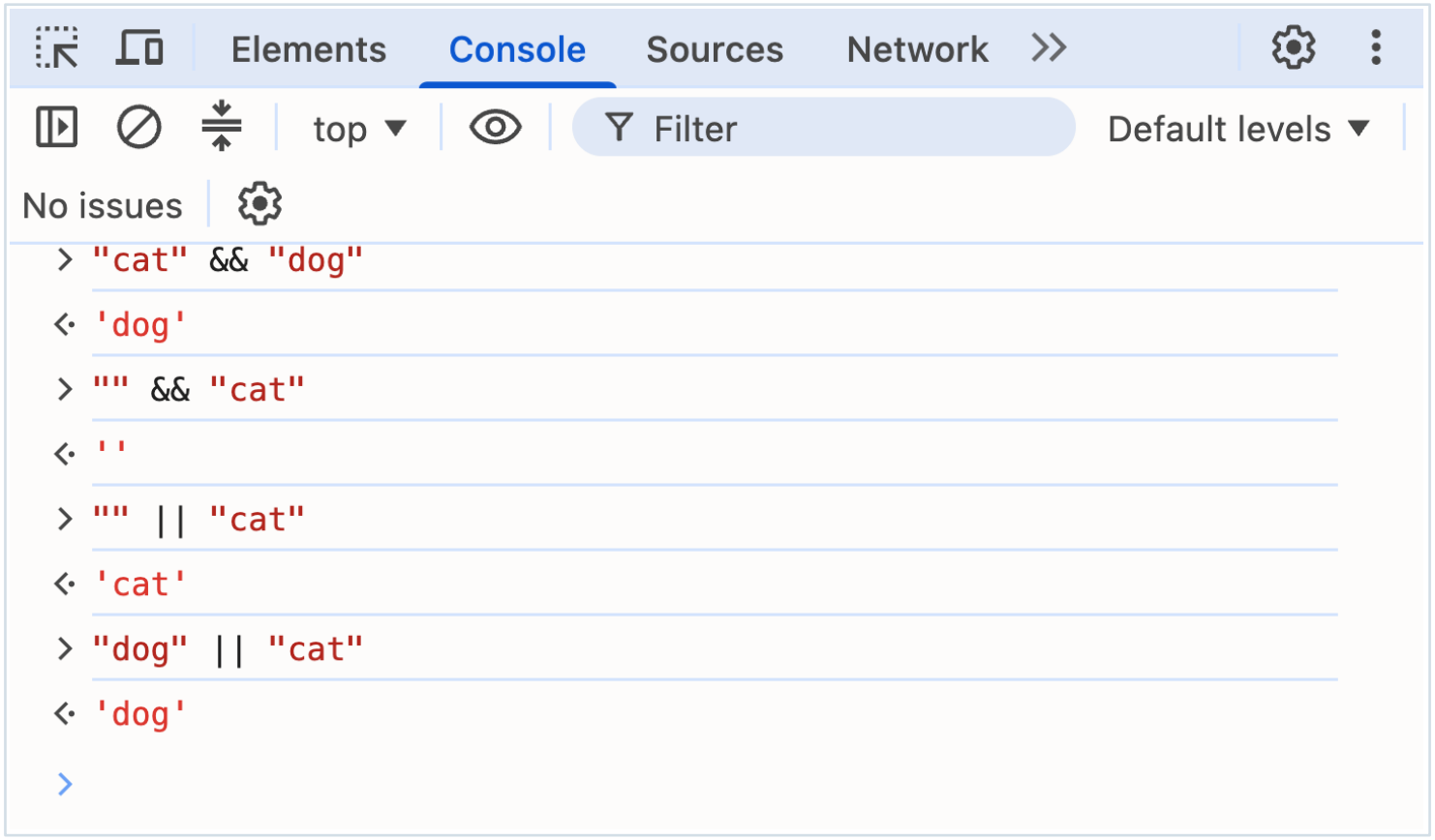
비교가 먼저

age >= 20 을 먼저 계산하고
그 결과를 && 로 묶습니다.

! 는 괄호와 함께

!true && false 는
(!true) && false 로 읽힙니다.
전체를 뒤집으려면 괄호.

값 하나를 돌려주는 && 와 ||



&& 는 거짓을 찾습니다

앞이 거짓이면 앞의 값,
아니면 뒤의 값을
돌려줍니다.

|| 는 참을 찾습니다

앞이 참이면 앞의 값,
아니면 뒤의 값을
돌려줍니다.

비어 있으면 기본값 — || 의 대표적인 쓰임

```
const nickname = "";
const name = nickname || "손님";
console.log(name);          // 손님 ← "" 는 거짓으로 취급

const nickname2 = "오즈";
const name2 = nickname2 || "손님";
console.log(name2);         // 오즈
```

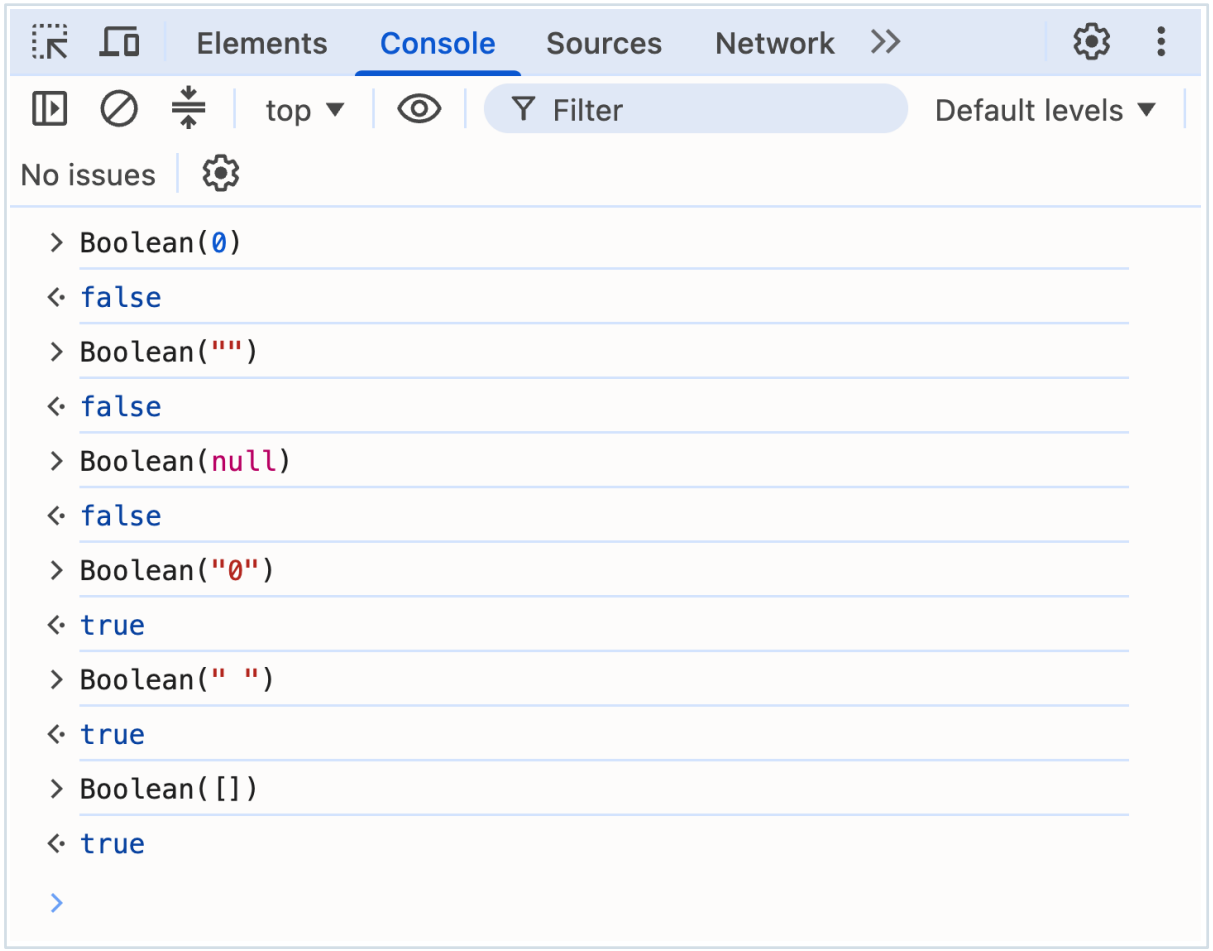
앞이 비었으면 뒤

입력이 비어 있을 때
대신 쓸 값을 뒤에
적어 둡니다.

실습 01에서 씁니다

이름이 비어 있으면
"회원"으로 보여 주는
코드를 만듭니다.

거짓으로 취급되는 정해진 값



falsy

false, 0, "" (빈 문자열),
null, undefined, NaN.
이 밖에 -0과 0n도 있습니다.

truthy

나머지는 모두 참으로
취급됩니다. "0", " ",
빈 배열 [] 도 참입니다.

여기까지 정리

1	**비교**	=== · !== 를 쓰고, 결과는 true 아니면 false
2	**논리**	&& 는 둘 다, || 는 하나만 참이면 참, ! 는 뒤집기
3	**돌려주는 값**	&& · || 는 피연산자 하나를 돌려줌 — || 로 기본값
4	**falsy**	false, 0, "", null, undefined, NaN 등 — 그 밖은 truthy

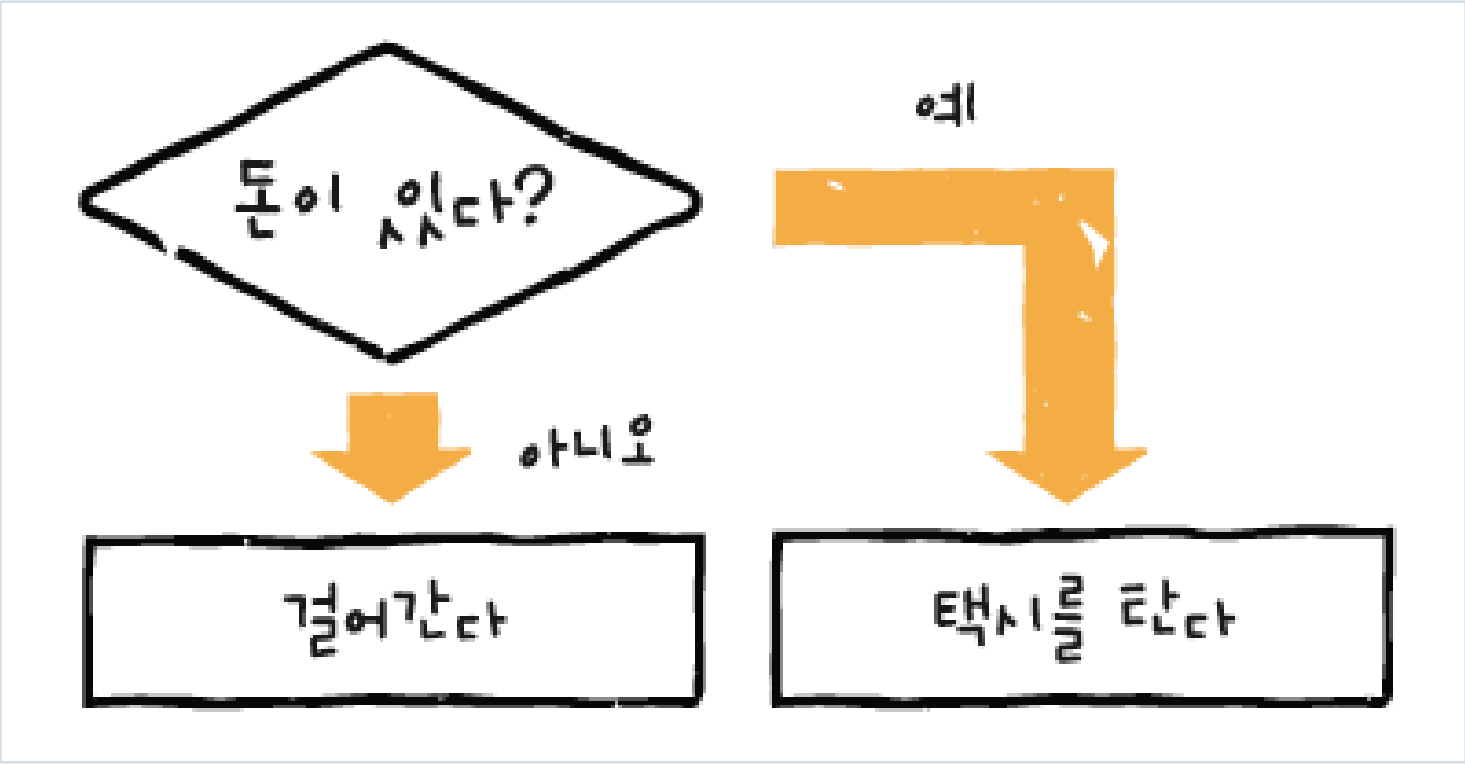
이제 이 참·거짓으로 갈 길을 나눕니다

03

조건문

if · else · else if · 조건 묶기 · 중첩 · switch · 실습 01

조건에 따라 나뉘는 갈 길



「돈이 있다?」에 예면 택시를 타고, 아니오면 걸어가는 순서도

조건

예 · 아니오로 답할 수 있는 질문입니다. 코드에서는 true · false가 나오는 식.

갈림길

참이면 이 길, 거짓이면 저 길. JavaScript에서는 if 문으로 씁니다.

if — 조건이 참일 때만 실행

```
// 모양
if (조건식) {
  // 조건식이 참이면 실행
}

// 예
const num = 10;
if (num % 2 === 0) {
  console.log("짝수입니다");
}
// 짝수입니다
```

거짓이면 건너뜁니다

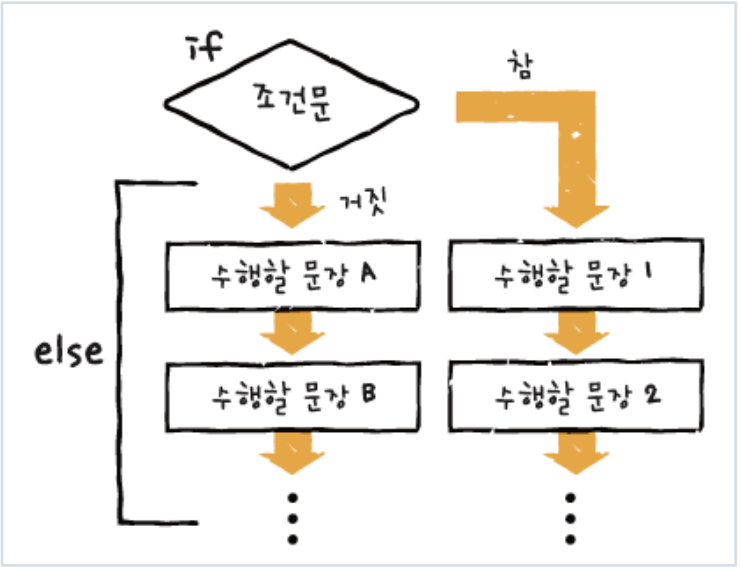
조건식이 false면
{ } 안을 건너뛰고
다음 줄로 갑니다.

괄호와 중괄호

조건식은 ( ) 안에,
실행할 코드는 { } 안에.
{ } 는 생략하지 않습니다.

else — 거짓일 때 갈 길

```
const num = 5;
if (num % 2 === 0) {
  console.log("짝수입니다");
} else {
  console.log("홀수입니다");
}
// 홀수입니다
```



참이면 오른쪽, 거짓이면 else 쪽

if와 else 가운데 반드시 하나만 실행됩니다

else if — 여러 갈래로 나누기

```
const num = 0;
if (num > 0) {
  console.log("양수");
} else if (num < 0) {
  console.log("음수");
} else {
  console.log("0");
}
// 0
```

위에서부터 차례로

처음으로 참인 갈래 하나만
실행하고 나머지는
건너뜁니다.

모두 아니면 else

위의 조건이 모두 거짓일 때
마지막 else가 실행됩니다.
else는 없어도 됩니다.

범위는 &&, 여러 값 가운데 하나는 ||

```
const score = 95;
if (score >= 90 && score <= 100) {   // 90 이상이면서 100 이하
  console.log("A 학점");
}

const day = "토";
if (day === "토" || day === "일") {   // 토요일이거나 일요일
  console.log("주말입니다");
}
```

사이의 값은 &&

90 <= score <= 100 처럼 이어 쓰지 않고 두 비교를 && 로 묶습니다.

양쪽 모두 비교식

day === "토" || "일" 은 늘 참이 됩니다. 섹션 04에서 자세히.

03 · truthy

if 안에 값만 넣는 경우

```
const userName = "";
if (userName) {
  console.log(`${userName}님, 안녕하세요`);
} else {
  console.log("이름을 입력해 주세요");
}
// 이름을 입력해 주세요 ← "" 는 falsy
```

값이 있으면 참

이름이 채워져 있으면
truthy라서 if 쪽으로
갑니다.

=== true 는 빼고

if (isMember === true) 대신
if (isMember) 로 씁니다.
반대는 if (!isMember).

03 · 중첩

if 안의 if

```
const isLoggedIn = true;
const role = "admin";

if (isLoggedIn) {
  if (role === "admin") {
    console.log("관리자 메뉴");
  } else {
    console.log("내 정보");
  }
} else {
  console.log("로그인 하기");
}
```

바깥이 참일 때만

안쪽 if는 바깥 조건이
참일 때만 검사합니다.
들여쓰기로 구분합니다.

&& 로 펼 수도

두 조건을 && 로 묶어
else if로 이어 쓰면
들여쓰기가 줄어듭니다.

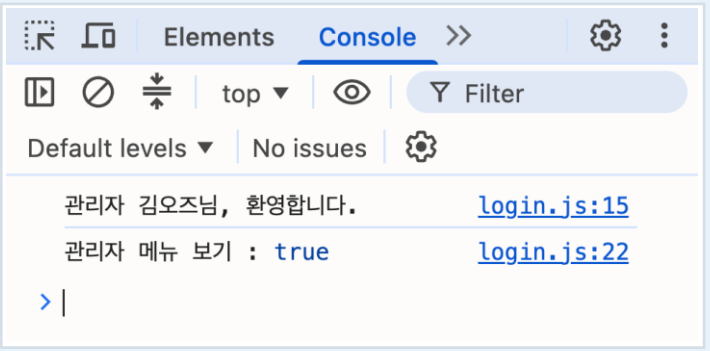
실습 01

로그인 상태에 따라 인사하기

login.js 맨 위의 세 값(로그인 여부 · 이름 · 역할)에 따라 다른 인사를 출력합니다.
값을 하나씩 바꿔 새로고침하며 네 가지 경우를 모두 확인합니다.

```
const isLoggedIn = true;
const userName = "김오즈";
const role = "admin";      // "admin" 또는 "member"
const displayName = userName || "회원";

if (!isLoggedIn) {
  console.log("로그인이 필요합니다.");
} else if (role === "admin") {
  console.log(`관리자 ${displayName}님, 환영합니다.`);
} else {
  console.log(`${displayName}님, 환영합니다.`);
}
```



처음 값 그대로 실행한 결과

값을 바꿔 보기

role → "member" : 일반 인사
userName → "" : 회원님
isLoggedIn → false : 로그인 안내

switch — 정해진 값 가운데 하나 고르기

```
const food = "멜론";
switch (food) {
  case "멜론":
    console.log("과일");
    break;
  case "당근":
    console.log("채소");
    break;
  default:
    console.log("과일도 채소도 아닙니다");
}
// 과일
```

( ) 의 값과 같은 case

food와 같은 case를 찾아
그 아래를 실행합니다.
비교는 === 로 합니다.

break · default

break에서 빠져나가고,
맞는 case가 없으면
default를 실행합니다.

break가 없으면 아래 case까지 이어서 실행

```
const food = "사과";
switch (food) {
  case "멜론":              // break가 없어 아래로 이어집니다
  case "사과":
    console.log("과일");
    break;
  case "당근":
    console.log("채소");
    break;
  default:
    console.log("과일도 채소도 아닙니다");
}
// 과일 — 멜론이어도 사과여도
```

일부러 묶을 때

case 여러 개를 붙여 쓰면
같은 코드를 함께
실행합니다.

빠뜨리면 실수

break를 빠뜨리면 다음
case의 코드까지 실행돼
결과가 바뀝니다.

범위는 if, 정해진 값 몇 개는 switch

```
// 90부터 99까지 — if
const score = 90;
if (score >= 90 && score < 100) {
  console.log("A++ 학점");
}
```

```
// 같은 일을 switch로
switch (score) {
  case 90:
  case 91:
  // … 92부터 98까지 생략
  case 99:
    console.log("A++ 학점");
    break;
}
```

범위를 검사할 때는 if, 메뉴처럼 정해진 값을 고를 때는 switch

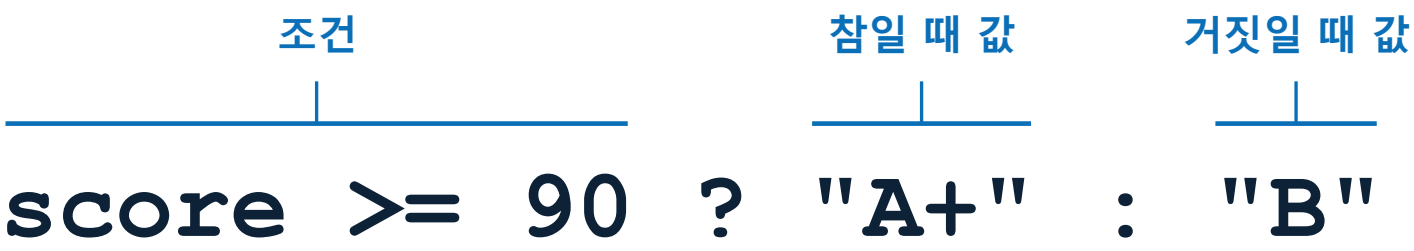
switch는 case에 적은 값과 같은지만 봅니다. 90 이상 같은 범위는 if로 씁니다.

04

삼항과 오류 점검

삼항 연산자 · 조건식 실수 다섯 가지 · 실습 02

삼항 연산자 — 값 하나 고르기



조건이 참이면 ? 뒤의 값, 거짓이면 : 뒤의 값

const grade = score >= 90 ? "A+" : "B"; 처럼 고른 값을 변수에 담을 때 씁니다.

값을 고를 때 더 짧은 삼항

```
// if…else로 쓰면
let result;
if (score >= 60) {
  result = "합격";
} else {
  result = "불합격";
}

// 삼항 연산자로 쓰면
const result2 = score >= 60 ? "합격" : "불합격";
console.log(`${isMember ? "회원" : "비회원"}입니다`);
```

값을 고를 때

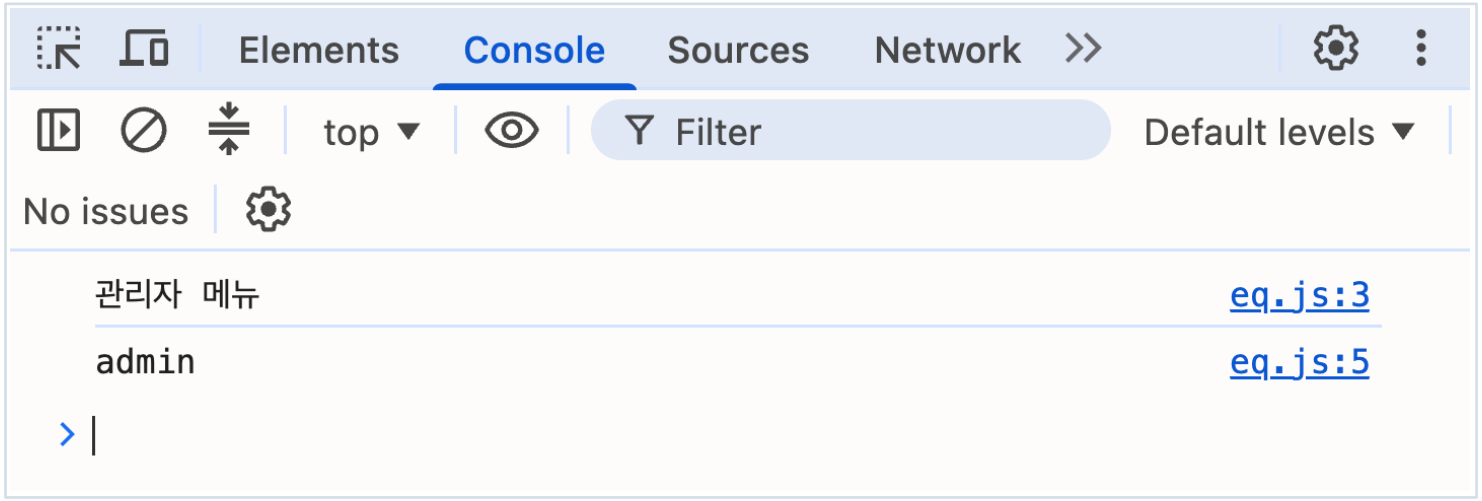
변수에 담을 값, 문장에
넣을 값을 고를 때
씁니다.

길어지면 if로

삼항 안에 삼항을 넣으면
읽기 어렵습니다.
갈래가 많으면 if.

비교할 자리에 쓴 =

```
let role = "member";
if (role = "admin") {        // = 는 '담기'
  console.log("관리자 메뉴");
}
console.log(role);
```



늘 참이 됩니다

role에 "admin"을 담고,
담은 값 "admin"이 truthy라
if 안이 실행됩니다.

같은지는 ===

if (role === "admin")
처럼 = 를 세 개 씁니다.
const면 TypeError가 납니다.

04 · 실수 2

먼저 검사한 넓은 조건

```
const score = 95;
let grade;
if (score >= 80) {            // 95도 여기서 먼저 걸립니다
  grade = "B";
} else if (score >= 90) {     // 여기까지 오지 않습니다
  grade = "A";
}
console.log(grade);           // B
```

처음 맞는 하나만

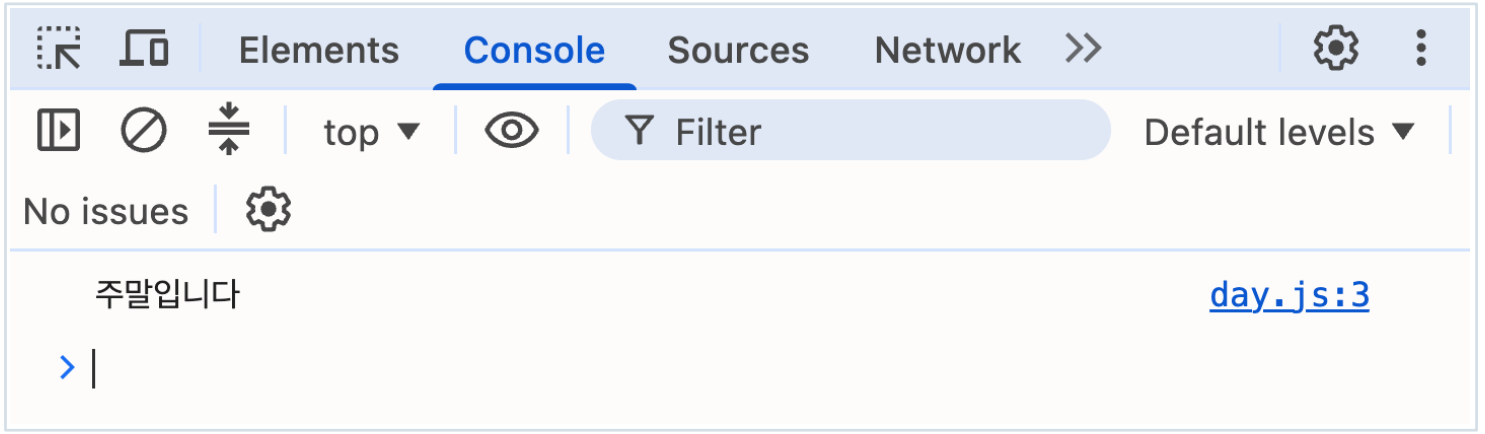
else if는 위에서부터 검사해 처음 참인 갈래만 실행합니다.

높은 기준부터

>= 로 범위를 나눌 때는 큰 기준(90)부터 차례로 검사합니다.

|| 뒤에 값만 적은 조건

```
const day = "월";
if (day === "토" || "일") {          // 월요일인데도 실행됩니다
  console.log("주말입니다");
}
if (day === "토" || day === "일") {  // 고친 코드
```



"일" 은 truthy

앞의 비교가 false라도
|| 가 "일"을 돌려주고,
"일"은 참으로 취급됩니다.

양쪽 모두 비교식

|| 와 && 의 양쪽에는
각각 비교식을
적습니다.

04 · 실수 4

숫자와 숫자처럼 보이는 문자열의 비교

```
console.log(10 < 9);       // false
console.log("10" < "9");   // true  ← 문자열끼리는 앞 글자부터
console.log("10" < 9);     // false ← 한쪽이 숫자면 숫자로 비교

const input = "2";
switch (input) {
  case 2:                  // "2" === 2 는 false
    console.log("2번 메뉴");
    break;
  default:
    console.log("없는 메뉴");
}
// 없는 메뉴
```

문자열은 한 글자씩

"10"과 "9"는 첫 글자
"1"과 "9"부터 비교해서
"10"이 작다고 봅니다.

Number()로 바꾸고

문자열로 받은 숫자는
Number()로 바꾼 뒤
비교합니다.

04 · 실수 5

빠진 중괄호, if 뒤에 붙은 ;

```
const score = 50;

if (score >= 90)
  console.log("A");
  console.log("우수");      // 조건과 상관없이 실행 — 우수

if (score >= 90); {        // ; 에서 if가 끝납니다
  console.log("A");        // 조건과 상관없이 실행 — A
}
```

중괄호를 씁니다

중괄호가 없으면 if 바로 다음 한 문장만 if에 속합니다.

if ( ) 뒤에는 ; 없이

; 를 붙이면 if가 끝나고, 뒤의 { } 는 따로 늘 실행됩니다.

04 · 점검

결과가 이상할 때 보는 순서

1	**조건식 찍기**	console.log(score >= 90) 처럼 조건식만 따로 찍어 true · false 확인
2	**= 인가 ===**	비교할 자리에 = 하나만 쓰지 않았는지
3	**순서**	else if는 위에서부터 — 넓은 조건이 먼저 나오지 않았는지
4	**양쪽 · break**	|| · && 양쪽에 비교식이 있는지, case 끝에 break가 있는지
5	**경계값**	90, 89, 60, 59, 0, 100처럼 경계의 값을 넣어 확인

빨간 줄이 없어도 틀릴 수 있습니다 — 값을 바꿔 가며 확인합니다

실습 02

점수 등급 계산기

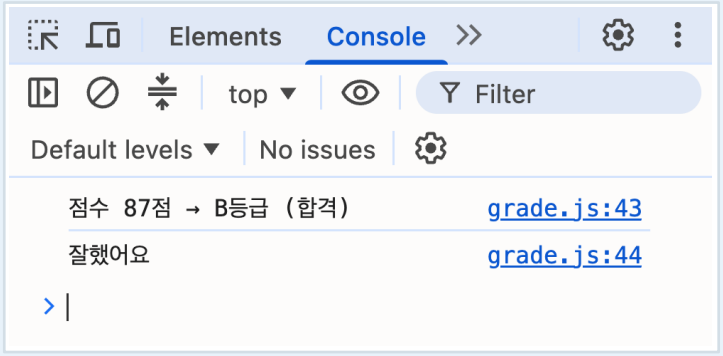
grade.js에 점수를 넣으면 등급과 합격 여부, 한마디를 출력합니다.
else if로 등급을, switch로 한마디를, 삼항 연산자로 합격 여부를 고릅니다.

해야 할 것

0~100 밖 점수는 || 로 먼저 거르기
90 · 80 · 70 · 60점 기준 A~F 등급
등급마다 한마디는 switch로
60점 이상이면 합격 (삼항 연산자)
「점수 87점 → B등급 (합격)」 출력

넣어 볼 점수

100 → A, 90 → A, 89 → B
60 → D 합격, 59 → F 불합격
0 → F, -5 · 105 → 안내 문구
C와 D는 같은 한마디
빨간 줄이 하나도 없다



87점으로 실행한 결과

다 되면 커밋합니다 — 이어서 일부러 조건을 틀려 봅니다

실습 02

일부러 조건을 틀려 보기

완성한 grade.js를 한 곳씩 바꿔 저장하고, 결과가 어떻게 틀리는지 본 뒤 되돌립니다.
세 가지 모두 빨간 줄이 뜨지 않습니다. 결과를 읽고 원인을 찾습니다.

바꿀 곳	넣을 점수	틀린 결과
① 등급 검사 순서를 거꾸로 — score >= 60(D)을 맨 처음에	87	D등급 (합격) · 조금만 더 힘내요
② 첫 if의 || 를 && 로	-5	F등급 (불합격) · 다시 도전해요
③ case "A" 끝의 break 지우기	95	A등급 (합격) · 잘했어요

세 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 02 끝

점검

조건문에서 자주 나오는 세 가지

!

조건이 늘 참이에요

= 를 하나만 썼거나,
|| 뒤에 비교식 없이
값만 적었습니다.

!

등급이 낮게 나와요

넓은 조건(>= 60)을
먼저 검사했습니다.
높은 기준부터 차례로.

!

한마디가 다르게 나와요

case 끝의 break를
빠뜨려 아래 case까지
이어서 실행됐습니다.

조건문 오류는 대부분 빨간 줄 없이 결과만 틀립니다

조건식만 따로 찍어 보고, 경계의 값을 넣어 보면 원인이 보입니다.

WRAP UP

오늘 정리

개념

연산자는 값을 받아 새 값을 만든다

+ 는 문자열을 이어 붙이고, - * / 는 숫자로

같은지는 ===, 결과는 true 아니면 false

&& · || 는 피연산자 하나를 돌려준다

falsy는 false, 0, "", null, undefined, NaN 등

else if는 처음 맞는 하나만 실행한다

직접 쓸 수 있어야 하는 것

% 로 짝수 판별하기

+= 로 합계 쌓기

&& · || 로 조건 묶기

if · else if · else로 나누기

switch와 break 쓰기

삼항 연산자로 값 고르기

참고교재

교재에서 함께 읽을 곳

오늘 수업과 이어지는 장입니다. 복습할 때 장 번호로 찾아 읽습니다.

길벗캠퍼스

『백엔드 입문자를 위한 모던 자바스크립트 & Node.js』

이창현 지음

4장. 재료들의 조합, 연산자
5장. 논리적 사고, 조건문

한빛미디어

『우아한 타입스크립트 with 리액트』

우아한형제들 웹프론트개발그룹 지음

2장. 타입

위키북스

『모던 자바스크립트 Deep Dive』

이웅모 지음

7장. 연산자
8장. 제어문

NEXT

DAY 12

반복문과 함수

같은 일을 여러 번 하는 반복문과
코드를 묶어 이름을 붙이는 함수를 배우고
구구단과 합계 함수를 만듭니다.

수고하셨습니다.